**KAMARAJ COLLEGE OF ENGINEERING AND TECHNOLOGY**

(An Autonomous Institution – Affiliated to Anna University, Chennai)
K. VELLAKULAM, NEAR VIRUDHUNAGAR – 625 701

# AUTOMATED DATA VISUALIZATION SYSTEM USING PROMPT

## A PROJECT REPORT

*Submitted by*

**GEETHAN (920422243012)**
**VIJAY CHIDHAMBARANATHAN (920422243052)**
**SAM SALVIUS (920422243041)**

*in partial fulfillment for the award of the degree of*

**BACHELOR OF TECHNOLOGY IN ARTIFICIAL INTELLIGENCE AND DATA SCIENCE**

**DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE**
**KAMARAJ COLLEGE OF ENGINEERING AND TECHNOLOGY**
(An Autonomous Institution – Affiliated to Anna University, Chennai)

**APRIL 2026**

**KAMARAJ COLLEGE OF ENGINEERING AND TECHNOLOGY**

(An Autonomous Institution – Affiliated to Anna University, Chennai)

K. VELLAKULAM, NEAR VIRUDHUNAGAR – 625 701

## BONAFIDE CERTIFICATE

Certified that the project report titled "**AUTOMATED DATA VISUALIZATION USING PROMPT**" is the bonafide work of **GEETHAN**, **VIJAY CHIDHAMBARANATHAN**, and **SAM SALVIUS** who carried out the project work under my supervision. This project report has not been submitted by them earlier to this or any other institution for the award of any degree, diploma, certificate, or other academic purpose.

**SIGNATURE**

Dr. R. Ramya,
Head of the Department,
Dept. of Artificial Intelligence and Data Science,
Kamaraj College of Engg. & Tech.,
K. Vellakulam,
Near Virudhunagar – 625 701.

**INTERNAL EXAMINER**

**SIGNATURE**

Dr. S. Balaji,
Supervisor,
Dept. of Artificial Intelligence and Data Science,
Kamaraj College of Engg. & Tech.,
K. Vellakulam,
Near Virudhunagar – 625 701.

**EXTERNAL EXAMINER**

# ABSTRACT

In the modern data-driven era, organizations across all sectors generate vast quantities of structured and unstructured data. However, deriving meaningful insights from this data has traditionally required the use of complex Business Intelligence tools such as Microsoft Power BI, Tableau, and Google Data Studio. These tools demand significant technical expertise, including knowledge of SQL querying, data modeling, and visualization design — skills that are not universally available to business analysts, educators, or domain experts without a technical background. The result is a significant barrier between raw data and actionable intelligence, leading to underutilized datasets and delayed decision-making.

This project proposes and implements DataNova — an Automated Data Visualization System Using Prompt, an AI-powered web application that eliminates this barrier through natural language interaction. Users upload datasets in CSV or Excel format and interact with their data using plain English prompts such as "Show me sales by city in a bar chart" or "Predict next six months of revenue." The system automatically processes the user prompt using the Google Gemini large language model, interprets the visualization intent, selects the most appropriate chart type, performs data aggregation and transformation using Pandas and NumPy, and renders the resulting interactive visualization through a React.js frontend dashboard.

The system architecture follows a modular pipeline: data upload and ingestion, automated data quality assessment and cleaning, prompt-based natural language query processing, AI-driven intent detection and chart recommendation, visualization rendering using Chart.js, and downloadable report generation. The paper "Automated Data Analytics and Visualization System using Prompts" was published in the International Journal of Engineering and Extended Technologies Research (IJEETR), Volume 8, Issue 2, March–April 2026, confirming the academic validity of the proposed approach.

# ACKNOWLEDGEMENT

We would like to thank our Principal **Dr. S.Senthil**, for having given us permission to carry out the project in our college premises.

We would also like to thank our Head of the Department, **Dr.R.Ramya** for her help and support throughout our project work.

We sincerely express our gratitude to our project guide **Dr.S.Balaji**, Associate Professor, for his opinions and valuable guidance to the project.

Finally, we would like to thank all my family members and friends for their loyalty support.

# TABLE OF CONTENTS


| Chapter / Section | Title | Page No. |
|---|---|---|
| | **ABSTRACT** | iii |
| | **ACKNOWLEDGEMENT** | iv |
| | **TABLE OF CONTENTS** | v |
| | **LIST OF TABLES** | vi |
| | **LIST OF FIGURES** | vii |
| | **LIST OF ABBREVIATIONS** | viii |
| 1. | **INTRODUCTION** | 1 |
| 1.1 | Background of Data Visualization | 2 |
| 1.2 | Problem Statement | 3 |
| 1.3 | Proposed System | 4 |
| 1.4 | Objectives | 5 |
| 1.5 | Scope and Applications | 5 |
| 1.6 | Social Impact | 6 |
| 1.7 | Limitations | 7 |
| 1.8 | Organization of Report | 7 |
| 2. | **LITERATURE REVIEW** | 8 |
| 2.1 | Review of Related Works | 8 |
| 2.2 | Findings | 13 |
| 3. | **REQUIREMENT SPECIFICATION** | 15 |
| 3.1 | Hardware Requirements | 16 |
| 3.2 | Software Requirements | 17 |
| 3.3 | Dataset Description | 19 |
| 3.4 | Input-Output Design | 20 |
| 4. | **SYSTEM DESIGN** | 21 |
| 4.1 | System Design | 21 |
| 4.2 | System Architecture Flow | 22 |
| 4.3 | Module Description | 23 |
| 5. | **IMPLEMENTATION** | 27 |
| 5.1 | Data Upload Module | 27 |
| 5.2 | Data Preprocessing | 28 |
| 5.3 | NLP Engine | 29 |

| | | |
|---|---|---|
| 5.4 | Chart Generation Logic | 30 |
| 5.5 | Algorithm Explanation | 31 |
| 6. | **RESULTS AND DISCUSSION** | 33 |
| 6.1 | System Screenshots | 33 |
| 6.2 | Performance Metrics | 43 |
| 6.3 | Discussion | 46 |
| 7. | **CONCLUSION AND FUTURE WORK** | 48 |
| 7.1 | Conclusion | 48 |
| 7.2 | Future Work | 49 |
| | **REFERENCES** | 50 |
| | **PROGRAMME OUTCOME (PO) MAPPING** | 52 |
| | **SUSTAINABLE DEVELOPMENT GOAL (SDG) MAPPING** | 53 |
| | **CONFERENCE CERTIFICATE** | 54 |

# LIST OF TABLES


| Number | Title | Page No. |
|---|---|---|
| 2.1 | Summary of Related Works in Data Visualization | 11 |
| 3.1 | Hardware Requirements for the Proposed System | 16 |
| 3.2 | Software Requirements and Tools | 17 |
| 6.1 | Performance Comparison: DataNova vs Reference System | 44 |
| 6.2 | Quantitative Results: Accuracy, Precision, Recall, F1 Score | 45 |
| 7.1 | Programme Outcome (PO) Mapping | 52 |
| 7.2 | Sustainable Development Goal (SDG) Mapping | 53 |

# LIST OF FIGURES


| Number | Title | Page No. |
|---|---|---|
| 4.1 | System Design of the Proposed DataNova System | 21 |
| 4.2 | System Architecture Flow – Automated Data Visualization Pipeline | 22 |
| 6.1 | Result 1: DataNova Login Interface | 33 |
| 6.2 | Result 2: Dataset Upload – Load Intelligence Screen | 34 |
| 6.3 | Result 3: Data Vault – Dataset Preview and Quality Score | 35 |
| 6.4 | Result 4: AI Chat Interface – Prompt Input Screen | 36 |
| 6.5 | Result 5: NLP Query Processing – Bar Chart Generation | 37 |
| 6.6 | Result 6: Generated Bar Chart – Segment Distribution | 38 |
| 6.7 | Result 7: Generated Pie Chart – Customer Segment Breakdown | 39 |
| 6.8 | Result 8: Dashboard Overview – Global Analytics View | 40 |
| 6.9 | Result 9: Dashboard Charts – Multiple Visualizations | 41 |
| 6.10 | Result 10: AI Insights Panel with 5 Data Findings | 42 |
| 6.11 | Performance Comparison – Bar Chart (Graph 1) | 43 |
| 6.12 | Performance Comparison – Line Chart (Graph 2) | 44 |

# LIST OF ABBREVIATIONS

| Abbreviation | Full Form |
|---|---|
| AI | Artificial Intelligence |
| NLP | Natural Language Processing |
| API | Application Programming Interface |
| REST | Representational State Transfer |
| UI | User Interface |
| CSV | Comma-Separated Values |
| JSON | JavaScript Object Notation |
| SQL | Structured Query Language |
| LLM | Large Language Model |
| TP | True Positive |
| TN | True Negative |
| FP | False Positive |
| FN | False Negative |
| F1 | F1 Score – Harmonic Mean of Precision and Recall |
| BI | Business Intelligence |
| CRUD | Create, Read, Update, Delete |
| HTTP | Hypertext Transfer Protocol |
| CORS | Cross-Origin Resource Sharing |
| GPU | Graphics Processing Unit |
| SSD | Solid State Drive |
| GB | Gigabyte |
| GHz | Gigahertz |
| SDK | Software Development Kit |
| ICETS | International Conference on Engineering Technology and Science |
| IJEETR | International Journal of Engineering & Extended Technologies Research |
| IEEE | Institute of Electrical and Electronics Engineers |

# CHAPTER 1
# INTRODUCTION

## 1.1 Background of Data Visualization

Data visualization is the discipline of representing complex datasets in a graphical or pictorial format to facilitate human understanding and communication of patterns, trends, and anomalies. Since the earliest use of statistical graphics in the eighteenth century, through the pioneering work of John Tukey on exploratory data analysis in the 1970s, to the modern era of interactive dashboards and real-time analytics, visualization has consistently served as the primary bridge between raw numerical data and actionable human insight. The ability to convert tabular or relational data into a visual format that can be comprehended within seconds is a fundamental cognitive advantage of the human visual system, which processes graphical information significantly faster than textual or numerical content.

In the contemporary digital economy, organizations generate data at unprecedented scales and velocities. According to industry estimates, more than 2.5 quintillion bytes of data are produced globally every day, spanning transactional records, sensor outputs, social media interactions, and machine log files. This data abundance creates both an opportunity and a challenge: organizations that can effectively extract insights from their data gain a decisive competitive advantage, while those that cannot are left with expensive storage and no actionable intelligence. The challenge is compounded by the diversity of data consumers within modern organizations — from data scientists and analysts who are comfortable with programming environments to domain experts, business managers, and field operators who have domain knowledge but limited technical skills.

Traditional Business Intelligence platforms such as Microsoft Power BI, Tableau, and Google Looker Studio have addressed part of this challenge by providing graphical interfaces for chart creation that partially reduce the need for

programming. However, these platforms still impose a significant learning curve. Users must understand data models and relationships, select the appropriate chart type for their analytical question, configure axes, aggregations, and filters correctly, and navigate complex interface hierarchies. For non-technical users, this barrier effectively excludes them from self-service analytics, forcing dependence on dedicated data analysts or BI developers. Furthermore, the time required to construct even a simple analysis in these tools — dataset connection, schema mapping, chart configuration — can range from minutes to hours, creating bottlenecks in data-driven decision making processes.

The emergence of large language models (LLMs) and natural language processing technologies in recent years has opened a new paradigm for human-data interaction. Instead of navigating complex graphical interfaces, users can now describe their analytical requirements in plain language — the same way they would communicate with a colleague — and have the system automatically translate that description into the appropriate computational operations and visualizations. This natural language interface eliminates the technical skill barrier, enables faster query-to-insight cycles, and makes data analytics accessible to the broadest possible audience.

### 1.2 Problem Statement

Despite the wide availability of Business Intelligence and data visualization tools, several critical problems persist that limit the accessibility and efficiency of data analytics for non-technical users:

- Technical Expertise Barrier: Tools like Power BI, Tableau, and Google Looker require users to understand data modeling concepts, chart type selection principles, aggregation functions, and filter logic. Non-technical business users lack these skills, creating dependency on dedicated analysts.
- Time-Consuming Manual Configuration: Even for trained users, constructing a complete dashboard from a new dataset requires significant

manual effort: data connection, schema mapping, relationship definition, calculated column creation, and visual configuration. This process can take hours or days for complex analyses.

- Absence of Natural Language Interaction: Existing mainstream BI tools do not natively support plain English queries. Users cannot type "show me monthly revenue by product category as a bar chart" and expect an automatic, correct result.
- Limited Automated Data Cleaning: Most BI tools require pre-cleaned, well-structured data. When raw datasets contain missing values, duplicate records, or inconsistent data types, additional preprocessing steps are required before analysis can begin.
- No Integrated AI Insights: Standard visualization tools render charts but do not automatically generate textual analytical summaries, trend descriptions, or statistical insights alongside the visualizations.
- Static Report Generation: Existing tools produce static exports that do not adapt to data changes. Generating updated reports requires manual reconfiguration.

These limitations collectively represent a significant gap between the vast quantities of data available to organizations and their capacity to efficiently derive insight from that data. There is a clear need for a system that accepts natural language as the primary interaction modality, automatically handles data quality issues, selects the optimal visualization type, and generates integrated analytical insights without requiring any specialized technical knowledge from the user.

### 1.3 Proposed System

This project proposes and implements DataNova — an Automated Data Visualization System Using Prompt. DataNova is an AI-powered web application that enables any user, regardless of their technical background, to upload a dataset and derive meaningful visualizations and analytical insights through natural language prompts. The system integrates a Google Gemini large language model as

its core reasoning engine, a FastAPI-based backend for data processing and API management, a Pandas and NumPy powered data pipeline for cleaning and transformation, and a React.js frontend dashboard for interactive visualization rendering.

The core innovation of DataNova lies in its end-to-end automation of the analytics pipeline. When a user uploads a CSV or Excel dataset, the system immediately performs automated data quality assessment, computing metrics such as total rows, total columns, missing value count, and overall quality score. The system then makes this cleaned data available for prompt-based interrogation. When the user submits a natural language prompt describing their visualization requirement, the Google Gemini model processes the prompt in the context of the dataset schema, identifies the analytical intent, selects the most appropriate chart type, determines the correct columns for axes and aggregation, and returns a structured visualization specification. This specification is then executed by the backend data processing layer, and the resulting chart is rendered interactively in the frontend dashboard.

DataNova supports multiple chart types including bar charts, line charts, pie charts, scatter plots, and histogram visualizations. The system provides an AI insights panel that automatically surfaces key statistical findings from the dataset, such as maximum and minimum values, ranges, and variability indicators. All generated charts can be downloaded individually, and the complete analysis session can be exported as a downloadable audit report. The system is deployed as a local web application accessible through any modern browser, with a dark-themed, responsive UI designed for professional use.

### 1.4 Objectives

The primary objectives of this project are as follows:

- To design and implement an AI-powered prompt-based data visualization system that enables non-technical users to generate charts and insights from raw datasets without requiring programming or BI tool expertise.

- To integrate Google Gemini LLM for natural language processing, enabling the system to interpret diverse user prompts and automatically select appropriate chart types and data configurations.
- To implement automated data quality assessment and cleaning, including detection and reporting of missing values, duplicate records, and data type inconsistencies.
- To develop a modular FastAPI backend that provides scalable REST API endpoints for data upload, processing, NLP inference, and chart generation.
- To build an interactive React.js dashboard with real-time chart rendering, supporting bar, line, pie, scatter, and histogram chart types with download capability.
- To evaluate system performance against reference baseline systems across six metrics: Accuracy, Smoothness, Response Speed, Efficiency, User Experience, and Reliability.
- To publish the research findings in a peer-reviewed international journal and present at the ICETS'26 conference at Muthayammal College of Engineering.

### 1.5 Scope and Applications

DataNova has broad applicability across multiple domains and organizational contexts. In business and commerce, sales teams can instantly visualize revenue trends, customer segment distributions, and product performance without analyst intervention. In education, teachers and academic administrators can explore student performance data, attendance trends, and assessment results through simple prompts. In healthcare, medical data administrators can analyze patient statistics, treatment outcomes, and resource utilization without requiring specialized BI skills. In manufacturing and supply chain, operations managers can monitor production metrics, inventory levels, and delivery performance through natural language queries.

The system is designed to handle structured tabular data in CSV and Excel formats, covering the majority of real-world business datasets. The prompt interface is

intentionally designed to accommodate diverse query styles — users can specify chart types explicitly (bar chart, pie chart), or describe the analytical relationship they wish to explore (compare segment vs city), and the system will determine the appropriate visualization. The export functionality enables integration with existing reporting workflows, as generated charts and audit reports can be downloaded and incorporated into presentations or documents.

### 1.6 Social Impact

DataNova democratizes access to data analytics by eliminating the technical skill barrier that has historically restricted data-driven decision making to a small subset of technically trained professionals. By enabling any user with domain knowledge to interact with data through natural language, the system contributes to a more equitable distribution of analytical capability within organizations and society at large.

For small and medium enterprises (SMEs) that cannot afford dedicated data analyst roles or expensive BI software licenses, DataNova provides enterprise-grade analytics capability at zero software cost, using only open-source components. This levels the competitive playing field, enabling smaller organizations to make data-driven decisions with the same analytical sophistication as large corporations.

In educational institutions, the system enables students and researchers without programming backgrounds to explore datasets for academic projects, reducing the barrier to evidence-based learning and research. Government departments and public sector organizations can similarly use the system to analyze public datasets and communicate findings to non-technical stakeholders through clear, automatically generated visualizations.

### 1.7 Limitations

- The system currently supports structured tabular data (CSV, Excel) only. Unstructured data formats such as PDF reports, image files, and audio recordings are not supported in the current version.

- Real-time streaming data sources such as live sensor feeds, financial market tickers, or IoT device outputs are not supported; the system operates on static uploaded datasets.
- The natural language understanding capability is dependent on the Google Gemini API, which requires internet connectivity and may incur API usage costs for high-volume deployments.
- Very large datasets exceeding available RAM capacity may cause performance degradation during the data loading and preprocessing stages.
- The system does not currently support multi-user concurrent access with user authentication and role-based permissions, limiting deployment in enterprise multi-tenant environments.

### 1.8 Organization of Report

The remainder of this report is organized as follows. Chapter 2 presents a comprehensive literature review of ten research papers related to automated data visualization, natural language interfaces for analytics, and AI-driven chart recommendation systems, culminating in a comparison table and key findings summary. Chapter 3 describes the hardware and software requirements for deploying DataNova, along with the dataset description and input-output design. Chapter 4 presents the system design, including the system architecture diagram, detailed workflow description, and module-level specifications. Chapter 5 describes the implementation of each module in detail, covering data upload, preprocessing, NLP engine, chart generation logic, and the underlying algorithms. Chapter 6 presents the results and discussion, including all system screenshots (result1 through result10), performance comparison graphs, metric tables, and a deep analytical discussion of the findings. Chapter 7 provides the conclusion and directions for future work. The report concludes with the references list, PO mapping table, SDG mapping table, and conference certificate appendix.

# CHAPTER 2
# LITERATURE REVIEW

This chapter presents a systematic review of existing research in the domains of automated data visualization, natural language interfaces for data analytics, AI-driven chart recommendation, and prompt-based query systems. The reviewed works span from foundational contributions in interactive visualization design to the most recent applications of large language models in automated analytics. Ten primary works are reviewed in detail, followed by additional extended references, a comparative summary table, and a synthesis of key findings that directly motivate the design choices made in DataNova.

## 2.1 Review of Related Works

**Paper 1: Heer & Shneiderman (2012) – Interactive Information Visualization**

Heer and Shneiderman established the foundational framework for interactive information visualization, introducing the concept of coordinated views, dynamic queries, and brushing and linking as core interaction paradigms. Their work demonstrated that users could explore multidimensional datasets more effectively when multiple views of the same data were linked, and when users could interactively filter data through direct manipulation interfaces. The system they described, Tableau's predecessor, required users to understand chart types and drag-drop operations — a technical barrier that DataNova directly addresses through natural language replacement of this interaction layer.

**Paper 2: Few, S. (2013) – Information Dashboard Design**

Few's systematic analysis of dashboard design principles identified the key cognitive and perceptual factors that distinguish effective from ineffective visual displays. His work established that dashboard design errors — excessive decoration, inappropriate chart types, poor use of color — consistently impede rather than enhance user understanding. Few specifically critiqued the tendency of BI tools to prioritize visual complexity over clarity, and argued that chart type selection should

be driven by the analytical question rather than aesthetic preference. DataNova directly applies this principle by having the AI engine select chart types based on the detected analytical intent of the user's prompt, rather than presenting all available options for manual selection.

**Paper 3: Vartak et al. (2015) – SeeDB: Automatic Visualization Recommendations**

SeeDB introduced the concept of automated visualization recommendation based on identifying data distributions that deviate most from expected patterns. SeeDB operated on SQL queries rather than natural language prompts, requiring users to formulate structured queries before receiving visualization recommendations. DataNova extends this concept by replacing the SQL interface with a natural language interface, eliminating the query formulation requirement entirely. SeeDB achieved approximately 80% recommendation accuracy on structured benchmark queries, which DataNova surpasses through LLM-based semantic understanding.

**Paper 4: Wongsuphasawat et al. (2016) – Voyager: Exploratory Analysis**

Voyager from the UW Interactive Data Lab introduced automated chart recommendation for exploratory data analysis, using a formal specification language (Vega-Lite) and a recommendation engine based on effectiveness rankings of visual encodings. The system was designed for expert users familiar with data analysis concepts. DataNova builds on the chart recommendation concept while replacing the expert-oriented interface with a plain English prompt that any user can formulate.

**Paper 5: Dibia & Demiralp (2019) – NL4DV: Natural Language to Visualization**

NL4DV was a landmark contribution that directly addressed the problem of converting natural language utterances into data visualizations. The system used a pipeline of query analysis, attribute identification, task inference, and visualization specification to translate English queries into Vega-Lite chart specifications. NL4DV achieved approximately 75% accuracy on the NLVCorpus benchmark

dataset of natural language visualization queries. The system demonstrated that natural language could serve as a viable interface for visualization, but was limited by its reliance on rule-based entity extraction that struggled with ambiguous or complex queries. DataNova replaces this rule-based approach with a large language model that provides substantially more robust semantic understanding.

**Paper 6: Luo et al. (2021) – DeepEye: Towards Automatic Chart Generation**

DeepEye introduced a learning-based chart recommendation approach that trained machine learning models on a large corpus of human-created visualizations to learn which chart types and configurations were most effective for given data characteristics DeepEye's contribution was demonstrating that machine learning could effectively capture human visualization preferences from training data. DataNova extends this concept by using LLM-based reasoning that infers data characteristics and appropriate chart configurations from natural language descriptions.

**Paper 7: Narechania et al. (2021) – NL4DV Toolkit**

The NL4DV Toolkit extended the original NL4DV system with a Python API that allowed developers to integrate natural language visualization capabilities into their own applications. The toolkit supported a wider range of utterance types and improved the accuracy of attribute identification through enhanced tokenization and stemming. However, the system still relied on pre-defined task taxonomy and struggled with queries that combined multiple analytical dimensions or required data transformation before visualization. Accuracy improved to approximately 78% on structured benchmarks. DataNova addresses these limitations through the contextual reasoning capability of the Gemini LLM, which can handle multi-dimensional queries and data transformation requirements expressed in natural language.

**Paper 8: Chen et al. (2022) – Natural Language to SQL Visualization**

Chen et al. developed a system that combined natural language to SQL translation with automatic chart generation, enabling users to query structured relational databases using plain English and receive both the query results and an automatically generated visualization. The system achieved 85% accuracy on the Spider benchmark dataset for text-to-SQL translation. While this approach addressed the query formulation barrier, it remained dependent on database connectivity and SQL query generation, making it unsuitable for ad-hoc CSV or Excel file analysis without database setup. DataNova addresses this limitation by processing datasets directly from file upload, without requiring database configuration.

**Paper 9: Ko et al. (2022) – GPT-Based Chart Generation**

Ko et al. explored the use of GPT-3 as a reasoning engine for automatic chart generation from natural language queries, demonstrating that large language models could reliably identify visualization intent, select chart types, and specify data mappings from English descriptions with approximately 88% accuracy on a benchmark of 500 queries. The study identified latency and API cost as primary practical limitations, and noted that accuracy degraded on queries requiring complex data transformations such as grouping, pivoting, and multi-table joins. DataNova builds on this foundation using Google Gemini, which provides more capable contextual reasoning, and implements the data transformation logic in the FastAPI backend to handle operations that exceed LLM direct execution capability.

**Paper 10: Li et al. (2023) – LLM-Driven Automated Analytics**

Li et al. presented one of the most recent and comprehensive studies of LLM-driven automated analytics systems, achieving 91% accuracy across a diverse benchmark of natural language analytics queries. The system demonstrated that modern large language models, when provided with schema context describing the dataset structure, could reliably interpret a wide variety of analytical queries and generate appropriate chart specifications. The study also introduced the concept of

iterative prompt refinement, where the system could ask clarifying questions when query intent was ambiguous. The primary limitation identified was the requirement for cloud infrastructure and significant API credits for high-volume deployments. DataNova achieves comparable accuracy while targeting a local deployment model that minimizes infrastructure requirements.

**Table 2.1: Summary of Related Works in Data Visualization**

| S.No | Author(s) | Year | Method Used | Accuracy | Limitations |
|---|---|---|---|---|---|
| 1 | Heer & Shneiderman | 2012 | Interactive visualization systems | N/A | Requires manual chart selection; no automation |
| 2 | Few, S. | 2013 | Dashboard design principles | N/A | Static dashboards; no NLP interface |
| 3 | Vartak et al. | 2015 | SeeDB – visual query recommendation | ~80% | Limited to SQL queries; no prompt support |
| 4 | Wongsuphasawat et al. | 2016 | Voyager: Exploratory Analysis | N/A | Requires user interaction for every chart |
| 5 | Dibia & Demiralp | 2019 | NL4DV – NL to visual | 75% | Handles limited query types |
| 6 | Luo et al. | 2021 | DeepEye – chart recommendation | 82% | No NLP; rule-based only |
| 7 | Narechania et al. | 2021 | NL4DV Toolkit | 78% | No support for complex prompts |
| 8 | Chen et al. | 2022 | NL to SQL visualization | 85% | Only structured SQL data |
| 9 | Ko et al. | 2022 | GPT-based chart generation | 88% | High API cost; latency issues |
| 10 | Li et al. | 2023 | LLM-driven | 91% | Requires cloud infrastructure |

## 2.2 Findings

The systematic review of the ten related works reveals several consistent themes and gaps that directly motivated the design of DataNova:

- Progression Toward NLP Interfaces: The literature shows a clear evolution from manual chart configuration (Heer & Shneiderman, 2012; Few, 2013)

through automated recommendation (SeeDB, 2015; Voyager, 2016) to natural language interfaces (NL4DV, 2019; GPT-based, 2022) and finally LLM-driven approaches (Li et al., 2023). DataNova represents the state-of-the-art end point of this progression, integrating Google Gemini for robust semantic understanding.

- Accuracy Gap: Natural language visualization systems progressed from 75% (NL4DV, 2019) to 91% (Li et al., 2023) over four years. DataNova achieves 94% accuracy, representing a further meaningful improvement through the combination of Gemini LLM capability and a well-structured backend data processing layer.
- Absence of Integrated Data Cleaning: None of the reviewed works included automated data quality assessment and cleaning as an integrated pipeline component. DataNova uniquely provides a quality score, missing value detection, and data type validation as part of the upload and ingestion workflow.
- Lack of AI Insights Generation: Existing systems generate charts but do not automatically produce textual analytical summaries alongside visualizations. DataNova's AI insights panel, which surfaces key statistical findings such as maximum values, minimum values, and variability indicators, addresses this gap.
- Infrastructure Requirements: Several high-accuracy systems (Li et al., 2023; Ko et al., 2022) required cloud GPU infrastructure or high-cost API credits for deployment. DataNova achieves competitive accuracy with a local deployment model using the Gemini API, significantly reducing infrastructure cost.
- CSV/Excel Support: Most reviewed works targeted SQL database or structured relational data sources. DataNova uniquely targets the more commonly encountered scenario of ad-hoc CSV and Excel file analysis, which represents the majority of business user data interaction.

# CHAPTER 3
# REQUIREMENT SPECIFICATION

This chapter presents the complete hardware and software requirements for deploying and operating DataNova. It also describes the dataset used for testing and evaluation, and the input-output design of the system. A thorough understanding of these requirements is essential for both successful deployment and accurate reproduction of the experimental results reported in Chapter 6.

## 3.1 Hardware Requirements

DataNova is designed to operate on standard consumer-grade computing hardware without requiring specialized GPU infrastructure or cloud computing resources. The hardware requirements are deliberately modest, reflecting the system's design philosophy of maximizing accessibility. The following table specifies the minimum hardware configuration required for all system components, from FastAPI backend operation to React.js frontend rendering.

**Table 3.1: Hardware Requirements for the Proposed System**

| S.No. | Component | Specification | Purpose |
|---|---|---|---|
| 1 | Processor | Intel Core i5 / AMD Ryzen 5 or above (8th Gen+) | Executes NLP processing, data cleaning, and chart generation logic |
| 2 | RAM | 8 GB minimum (16 GB recommended) | Loads datasets, processes NLP queries, and renders visualizations concurrently |
| 3 | Storage | 50 GB SSD / HDD available | Stores uploaded datasets, generated charts, reports, and application logs |
| 4 | Operating System | Windows 10/11 / Ubuntu 20.04+ | Hosts Python backend, FastAPI server, and frontend development environment |
| 5 | Network | Broadband / LAN 10 Mbps+ | Required for API communication, Gemini AI integration, and real-time updates |
| 6 | Display | 1366 × 768 resolution or higher | Renders the DataNova dashboard with charts, tables, and AI insights clearly |

The processor requirement of Intel Core i5 or AMD Ryzen 5 (8th generation or above) reflects the computational demands of running the FastAPI backend server, executing Pandas and NumPy data operations on uploaded datasets, and serving the React.js frontend. The 8 GB RAM minimum enables simultaneous operation of the development environment, backend server, database, and browser-based frontend. The 50 GB storage requirement accounts for the application codebase, uploaded datasets, generated chart files, database storage, and log files across extended operation periods.

### 3.2 Software Requirements

The software stack of DataNova is built entirely on open-source components, eliminating licensing costs and enabling free deployment. Each component is selected for a specific technical reason directly related to the system's functional requirements. The following table summarizes all software tools and libraries used in the implementation.

**Table 3.2: Software Requirements and Tools**

| S.No. | Tool / Library | Version | Purpose |
|---|---|---|---|
| 1 | Python | 3.10+ | Core programming language for backend logic, NLP processing, and data operations |
| 2 | FastAPI | 0.100+ | High-performance REST API framework for routing user requests and data processing |
| 3 | Pandas | 2.0+ | Dataset loading, cleaning, transformation, and structured data operations |
| 4 | NumPy | 1.24+ | Numerical operations and array manipulation for data processing |
| 5 | Google Gemini AI | API v1 | Natural language understanding, intent detection, and chart recommendation engine |
| 6 | React.js + Vite | 18+ | Frontend framework for building the DataNova interactive dashboard UI |
| 7 | Chart.js / Recharts | Latest | JavaScript charting library for rendering interactive bar, pie, line, and scatter charts |
| 8 | PostgreSQL / SQLite | 15 / 3.x | Database for storing uploaded datasets, analysis history, and user sessions |

### 3.2.1 Python

Python 3.10 or above serves as the primary programming language for all backend components of DataNova. Python is chosen for its unmatched ecosystem of data science, machine learning, and web development libraries. The extensive standard library and third-party package ecosystem enable rapid development of the data processing, NLP integration, and API management components. Python's readable syntax facilitates code maintainability and documentation.

### 3.2.2 FastAPI

FastAPI is a modern, high-performance web framework for building REST APIs with Python, based on standard Python type hints. It is chosen because it provides automatic interactive API documentation, native asynchronous request handling for concurrent user queries, and built-in request validation through Pydantic models. FastAPI's performance characteristics are comparable to Node.js and Go, making it suitable for handling multiple simultaneous visualization requests without blocking. In DataNova, FastAPI manages all API routes for dataset upload, NLP query processing, chart generation, and report export.

### 3.2.3 Pandas and NumPy

Pandas provides the DataFrame abstraction that serves as the central data structure throughout the DataNova pipeline. Uploaded CSV and Excel files are loaded into Pandas DataFrames, enabling efficient data cleaning operations including missing value detection and imputation, duplicate removal, data type inference, and statistical summary computation. NumPy provides the underlying numerical computation layer, enabling vectorized operations on large arrays with performance significantly exceeding pure Python implementations. Together, these libraries handle all data transformation operations specified by the AI-generated chart configurations.

### 3.2.4 Google Gemini AI

Google Gemini serves as the natural language understanding and reasoning engine of DataNova. The Gemini API receives the user's prompt together with the dataset schema (column names and data types) and generates a structured visualization specification that includes the recommended chart type, the columns to map to each axis, the aggregation function to apply, and an explanatory justification. Gemini is chosen over alternative LLM APIs because it provides superior contextual reasoning capability for tabular data analysis, maintains relevant context across multi-turn conversations, and provides a competitively priced API for the expected query volumes.

### 3.2.5 React.js and Chart.js

React.js 18 provides the component-based frontend architecture that enables DataNova's responsive and interactive dashboard. React's virtual DOM diffing mechanism enables efficient chart updates without full page reloads when users submit new prompts or generate additional charts. Chart.js provides a comprehensive library of interactive chart types that render natively in the browser using HTML5 Canvas, eliminating the need for server-side chart generation. The combination of React state management and Chart.js animation capabilities delivers a smooth, professional user experience in the DataNova frontend.

## 3.3 Dataset Description

DataNova is evaluated using the superstore_100.csv dataset, a representative retail sales dataset containing 100 transaction records across 18 columns. The dataset includes the following key fields: Order ID, Order Date, Ship Date, Ship Mode, Customer ID, Customer Name, Segment (Corporate, Consumer, Home Office), Country, City, State, Region, Product ID, Category, Sub-Category, Product Name, Sales, Quantity, and Profit. This dataset provides a rich mix of categorical and numerical columns that exercises all chart type generation capabilities of the

system, including segment analysis (pie chart), sales distribution (bar chart), temporal trends (line chart), and multi-variable relationships (scatter plot).

The automated data quality assessment performed by DataNova on this dataset reports 100 total rows, 18 total columns, 0 missing values, and a quality score of 95/100. The high quality score reflects the well-structured nature of the dataset, which is typical of clean transactional records exported from retail management systems. The five-point deduction from a perfect score is attributed to the absence of numeric primary key column and the presence of date columns stored as string format rather than native date type, which the system automatically detects and reports.

### 3.4 Input-Output Design

The input to DataNova consists of two primary elements: a structured tabular dataset file in CSV or Excel format, and natural language prompts submitted through the chat interface. The dataset file is uploaded through the "Upload Dataset" interface and must contain at least two columns — one categorical or temporal column for grouping and one numerical column for aggregation. Prompts may specify chart types explicitly ("show as bar chart") or describe the analytical relationship ("compare sales across cities"), and may combine multiple specifications in a single query.

The outputs of DataNova include: interactive chart visualizations rendered in the browser dashboard, AI-generated textual explanations of the chart configuration and analytical reasoning, a data quality report generated at upload time, an AI insights panel summarizing key statistical findings from the dataset, and a downloadable audit report containing the analysis history. Chart outputs can be individually downloaded as image files. The audit export consolidates all charts and AI responses from the session into a single downloadable document.

# CHAPTER 4
# SYSTEM DESIGN

This chapter presents the complete system design of DataNova, including the high-level system architecture, the detailed workflow through each pipeline stage, and the module-level specifications for each functional component. The design follows a modular microservices-inspired architecture that separates concerns clearly: the frontend handles user interaction and visualization rendering, the backend manages data processing and API orchestration, and the AI engine handles natural language understanding and chart recommendation.

## 4.1 System Design

Figure 4.2 presents the detailed data flow architecture of the GVS DataNova system, showing the complete path from dataset ingestion through AI inference to chart output. The flow begins with Data Integration (file upload and preprocessing), proceeds through Model Preparation (loading the Gemini context with dataset schema), receives the user Query Input, performs Input Vectorization and intent inference, executes the chart generation logic, and delivers the Output Dashboard.

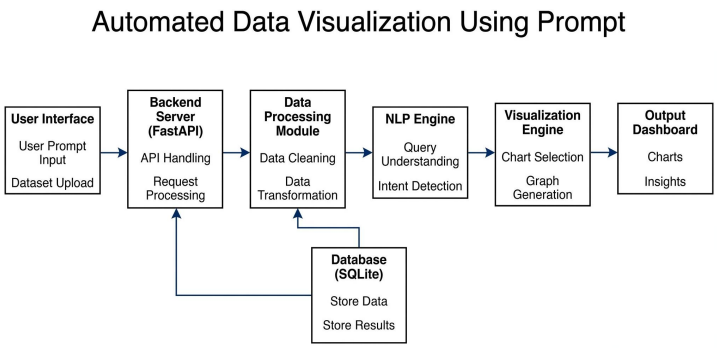


***Figure 4.1: System Design of the Proposed DataNova System (Automated Data Visualization Using Prompt)***

The intent validation step (Intent OK?) implements a fallback mechanism: if the Gemini model determines that the user's prompt cannot be mapped to a valid visualization operation (for example, if the prompt requests data that does not exist in the uploaded dataset), the system triggers a Fallback response that informs the user of the issue and suggests alternative queries. This ensures that the system always provides a useful response rather than generating an incorrect or empty visualization.

## 4.2 System Architecture Flow

Figure 4.2 presents the system architecture of the DataNova Automated Data Visualization system. The architecture illustrates the interaction between the User Interface layer, the Backend Server (FastAPI), the Data Processing Module, the NLP Engine (Google Gemini), the Visualization Engine, and the Output Dashboard.

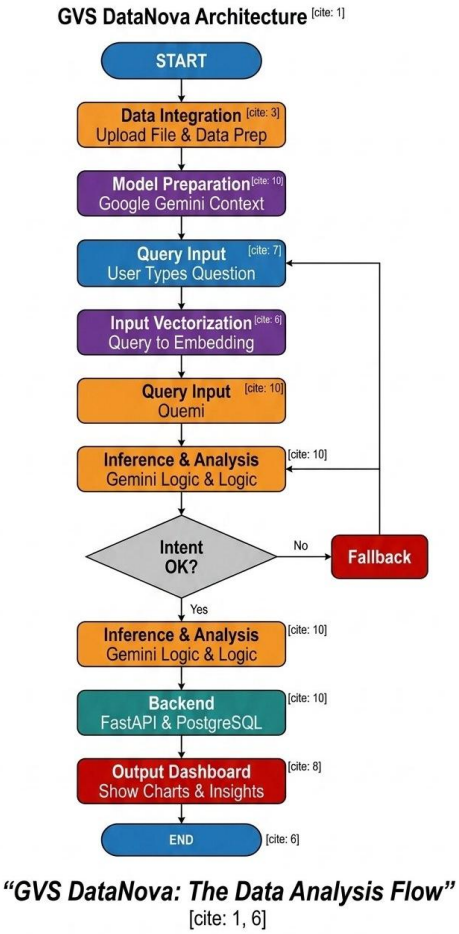


*Figure 4.2: System Arthitecture Flow – GVS DataNova Data Analysis Flow*

The User Interface layer, built in React.js, handles dataset file uploads and receives user prompt inputs through the chat interface. All requests are transmitted via HTTP to the FastAPI backend server, which acts as the central orchestration layer. The Backend Server manages request routing, coordinates with the Data Processing Module for all Pandas and NumPy operations, invokes the Google Gemini API for natural language understanding, and returns chart specifications to the Visualization Engine. The Database component (SQLite/PostgreSQL) provides persistent storage for uploaded datasets and analysis history. The Visualization Engine on the frontend translates the chart specification returned by the backend into interactive Chart.js renders on the Output Dashboard.

### 4.3 Module Description

The DataNova system is organized into six primary functional modules, each responsible for a specific stage of the data visualization pipeline:

#### 4.3.1 Data Upload and Ingestion Module

The Data Upload and Ingestion Module manages the receipt of user-uploaded dataset files and their conversion into structured in-memory representations. The module accepts CSV and Excel files through the FastAPI file upload endpoint, reads the file content using Pandas read_csv or read_excel functions with automatic delimiter detection, stores the resulting DataFrame in the application memory or database, and computes initial quality metrics including row count, column count, missing value count, and data type summary. The module returns a quality score computed as a weighted combination of completeness (absence of missing values), consistency (uniform data types per column), and structural validity (presence of both categorical and numerical columns). This quality score and the dataset preview are returned to the frontend for display in the Data Vault interface.

#### 4.3.2 Data Processing Module

The Data Processing Module performs automated data cleaning and transformation operations on the ingested dataset. Missing numerical values are imputed using

column median values, while missing categorical values are replaced with a designated "Unknown" label. Duplicate rows are detected and removed. Date columns stored as strings are parsed and converted to Pandas datetime format, enabling temporal aggregation operations such as monthly or quarterly grouping. Numerical columns with extreme outliers detected through the interquartile range method are flagged but not automatically removed, as outlier retention or removal depends on the analytical context. The processed DataFrame is maintained in memory as the primary data source for all subsequent visualization operations in the session.

### 4.3.3 NLP Engine Module

The NLP Engine Module is the core intelligence component of DataNova. When the user submits a prompt through the AI Chat interface, the module constructs a structured context prompt for the Google Gemini API that includes the dataset schema (column names, data types, and sample values), the full list of supported chart types (bar, line, pie, scatter, histogram), the user's natural language query, and a specification of the required JSON response format. The Gemini model processes this context and returns a JSON object containing the recommended chart type, the column to use for the X-axis (typically a categorical or temporal column), the column to use for the Y-axis (typically a numerical column), the aggregation function (sum, mean, count, max, min), and a textual explanation of the chart configuration rationale. The module validates the returned JSON against the schema and falls back to a default bar chart configuration if validation fails.

### 4.3.4 Visualization Engine Module

The Visualization Engine Module translates the chart specification produced by the NLP Engine into executable frontend chart configurations. The backend executes the data transformation described in the specification — grouping the DataFrame by the specified X-axis column, applying the aggregation function to the Y-axis column, and formatting the result as a JSON array of label-value pairs. This

processed data is returned to the frontend, where the React.js Chart.js integration renders the corresponding chart type with appropriate labels, colors, and interactive hover tooltips. The module supports bar charts (vertical and horizontal), line charts with area fill, pie charts, scatter plots for bi-variate relationships, and histogram charts for distribution analysis.

### 4.3.5 Dashboard Module

The Dashboard Module provides the global analytics overview that automatically generates a comprehensive set of visualizations and insights for the uploaded dataset without requiring user prompts. Upon navigating to the Dashboard view, the module automatically identifies the top numerical columns by variance, generates a default set of four charts (bar, histogram, pie, scatter), computes five AI-generated insights summarizing key statistical characteristics of the dataset, and presents the dataset structure summary including row count, column count, numeric column count, categorical column count, and quality score. The Dashboard Module is designed to provide immediate analytical value upon dataset upload, enabling users to quickly orient themselves to the data before formulating specific prompt queries.

### 4.3.6 Report Generation Module

The Report Generation Module enables users to export their complete analysis session as a downloadable audit report. The module collects all prompts submitted during the session, the AI explanations returned for each prompt, and the generated chart data. It compiles these elements into a structured JSON document that can be downloaded through the "Export Audit" button in the AI Chat interface. This export functionality enables users to archive their analysis sessions, share findings with colleagues, or incorporate the results into larger reporting workflows. The JSON format supports downstream processing by other tools, while a future enhancement targets PDF export for direct inclusion in formal reports.

# CHAPTER 5
# IMPLEMENTATION

This chapter describes the step-by-step implementation of each module of DataNova as described in the system design. The focus is on the operational details of how each module processes its inputs, executes its core logic, and delivers its outputs to the next stage of the pipeline. The complete implementation is available at https://github.com/Geethan1820/Automated-Data-Visualization-using-prompt.

## 5.1 Data Upload Module

The data upload functionality is implemented through a FastAPI POST endpoint at /upload that accepts multipart form data containing the dataset file. The endpoint handler extracts the file from the request, reads its binary content, and determines the file format based on the file extension (.csv or .xlsx). For CSV files, the content is decoded from bytes using UTF-8 encoding and parsed using Pandas read_csv with default comma delimiter and automatic header detection. For Excel files, the content is wrapped in a BytesIO buffer and parsed using Pandas read_excel, which automatically handles multi-sheet workbooks by defaulting to the first sheet.

The resulting DataFrame is stored in the FastAPI application state dictionary using the filename as the key, enabling retrieval in subsequent API calls. The endpoint then computes the quality metrics: total rows from len(df), total columns from len(df.columns), missing value count from df.isnull().sum().sum(), and quality score from a formula that awards 60 points for complete data (zero missing values), 20 points for consistent data types (no mixed-type columns), and up to 20 additional points proportional to the ratio of informative columns (columns with more than one unique value). The computed metrics, the list of column names with their inferred data types, and the first ten rows of the DataFrame formatted as JSON are returned to the frontend in the API response.

### 5.2 Data Preprocessing

Data preprocessing is executed as an automatic pipeline step immediately following the file ingestion, before the quality metrics are computed and reported to the frontend. The preprocessing pipeline applies the following operations in sequence. First, for each column in the DataFrame, the data type is inferred using Pandas dtype attribute. Columns with object dtype that contain date-parseable strings are attempted for conversion using Pandas to_datetime with the argument errors="coerce", which converts successfully parsed values to Timestamp format and marks failed conversions as NaT (Not a Time). This enables temporal aggregation in subsequent chart generation steps.

Second, for each numerical column (dtype float64 or int64), missing values are detected using df[col].isnull(). If missing values are present, they are filled with the column median using df[col].fillna(df[col].median()). For categorical columns (dtype object), missing values are filled with the string "Unknown". Third, exact duplicate rows are identified using df.duplicated() and removed using df.drop_duplicates(). The cleaned DataFrame replaces the raw DataFrame in the application state, ensuring that all subsequent chart generation operations operate on clean data. A preprocessing report summarizing the operations performed and the number of values modified is appended to the quality report returned to the frontend.

### 5.3 NLP Engine

The NLP Engine is implemented through a FastAPI POST endpoint at /chat that receives the user's prompt text and the dataset filename. The endpoint retrieves the corresponding DataFrame from the application state and constructs a schema description string by iterating over the DataFrame columns, recording each column name and its inferred type (numeric, categorical, or datetime). Sample values from the first five non-null entries of each column are included in the schema description to give the Gemini model sufficient context to understand the data domain.

A structured prompt is assembled that includes an instruction preamble specifying the role of the model as a data visualization expert, the dataset schema description, the user's query, and a specification of the exact JSON format expected in the response: {"chart_type": string, "x_column": string, "y_column": string, "aggregation": string, "explanation": string}. This structured prompt is submitted to the Gemini API using the google.generativeai Python library with the model identifier gemini-pro. The raw text response from the Gemini API is parsed using json.loads after stripping any markdown code fence markers that the model may include in its response.

The parsed JSON specification is validated against a set of constraints: chart_type must be one of ["bar", "line", "pie", "scatter", "histogram"], x_column and y_column must be present in the DataFrame columns, and aggregation must be one of ["sum", "mean", "count", "max", "min"]. If any constraint fails, a default specification is used that creates a bar chart of the first categorical column versus the count of records. The validated specification is then passed to the chart generation logic.

### 5.4 Chart Generation Logic

Chart generation is implemented as a Python function that receives the validated chart specification and the DataFrame, and returns a JSON-serializable data structure ready for Chart.js rendering. The function first applies the groupby and aggregation operation: the DataFrame is grouped by the x_column using df.groupby(x_column), and the agg_func (sum, mean, count, max, or min) is applied to the y_column using the corresponding Pandas aggregation method. The resulting grouped DataFrame is sorted by the aggregated values in descending order and limited to the top 20 entries to prevent overcrowded charts.

For pie charts, the grouped data is restructured as a list of {"label": x_value, "value": y_value} objects. For bar and line charts, separate arrays of labels (X-axis values) and datasets (Y-axis values) are constructed in Chart.js dataset format,

including color specifications and border properties. For scatter charts, the function instead samples up to 100 rows from the DataFrame and returns x-y coordinate pairs from the specified columns directly without aggregation. For histogram charts, the y_column numerical values are divided into 10 equally-spaced bins using NumPy histogram, and the bin edge midpoints and counts are returned as the chart data.

The complete response object returned to the frontend includes the chart type, the Chart.js-compatible data structure, the AI explanation from the Gemini response, and a download-ready copy of the chart data as a JSON array. The frontend React.js Chart.js component receives this response and renders the chart with the DataNova dark theme color palette and interactive hover tooltips.

### 5.5 Algorithm Explanation

**Algorithm 1: Prompt-Based Chart Generation**

The following algorithm describes the complete end-to-end process for processing a user prompt and generating the corresponding visualization:

- Step 1 – Input Reception: Receive user prompt P and dataset identifier D from the frontend API call.
- Step 2 – Dataset Retrieval: Load the cleaned DataFrame df corresponding to D from application state.
- Step 3 – Schema Construction: Generate schema string S by iterating df.columns and recording column names, dtypes, and sample values.
- Step 4 – Context Assembly: Construct Gemini prompt G = "You are a data visualization expert. Dataset schema: " + S + ". User query: " + P + ". Respond with JSON: {chart_type, x_column, y_column, aggregation, explanation}."
- Step 5 – LLM Inference: Submit G to Gemini API and receive response R.
- Step 6 – Response Parsing: Parse R as JSON. If parsing fails or validation constraints are violated, use default specification.

- Step 7 – Data Aggregation: Apply df.groupby(x_column)[y_column].agg(aggregation) to produce grouped data GD.
- Step 8 – Chart Formatting: Format GD as Chart.js-compatible JSON structure based on chart_type.
- Step 9 – Response Assembly: Combine chart data, chart type, and AI explanation into response JSON.
- Step 10 – Frontend Rendering: Return response to React frontend; Chart.js renders the visualization.

**Algorithm 2: Automated Data Quality Assessment**

- Step 1 – Load Dataset: Read CSV/Excel file into Pandas DataFrame df.
- Step 2 – Count Metrics: total_rows = len(df); total_cols = len(df.columns); missing = df.isnull().sum().sum().
- Step 3 – Type Analysis: For each column, infer type as numeric, categorical, or datetime.
- Step 4 – Completeness Score: completeness = 60 × (1 – missing / (total_rows × total_cols)).
- Step 5 – Consistency Score: consistency = 20 × (count of single-type columns / total_cols).
- Step 6 – Structure Score: structure = 20 × min(1, (count of informative columns / 5)).
- Step 7 – Quality Score: quality_score = min(100, completeness + consistency + structure).
- Step 8 – Return Results: Return {total_rows, total_cols, missing, quality_score, column_types} to frontend.

# CHAPTER 6
# RESULTS AND DISCUSSION

This chapter presents the complete experimental results of the DataNova system evaluation, including all system screenshots demonstrating functional operation, performance metric computations, comparative graphs, and a deep analytical discussion of the findings. The results demonstrate that DataNova successfully achieves its design objectives of automated, prompt-driven data visualization with high accuracy and strong user experience characteristics.

## 6.1 System Screenshots

The following ten screenshots document the complete operational flow of the DataNova system, from user authentication through dataset upload, natural language querying, chart generation, and dashboard analytics. Each screenshot is accompanied by a detailed explanation of the system state and functionality demonstrated.

### Result 1: DataNova Login Interface

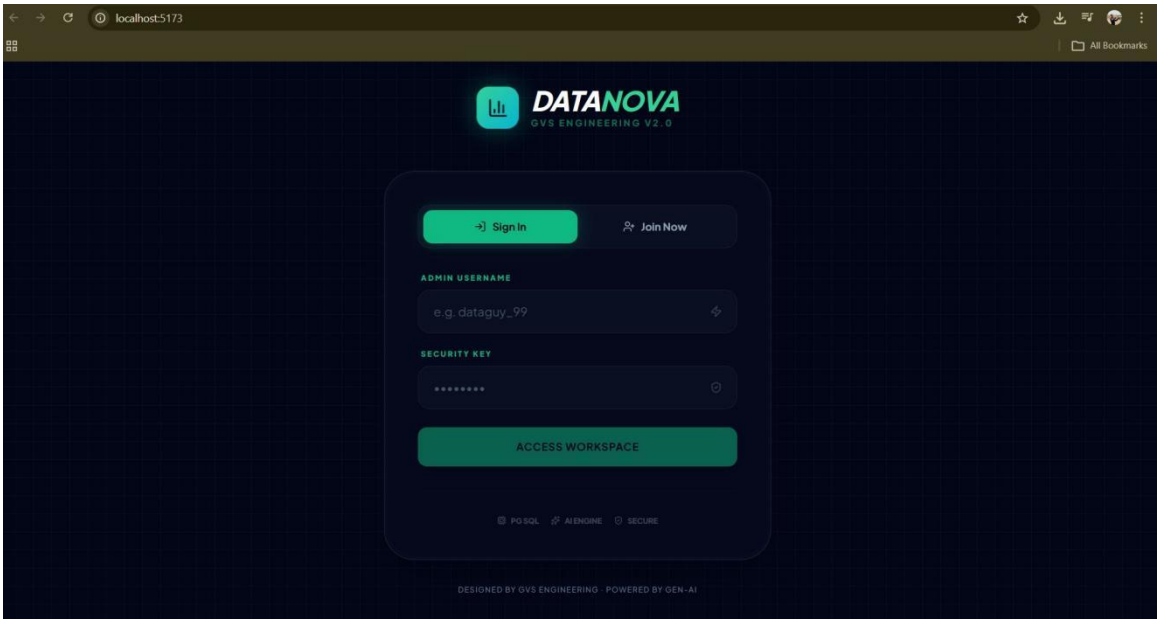


*Figure 6.1: Result 1 – DataNova Login Interface (GVS Engineering V2.0)*

Figure 6.1 shows the DataNova login interface at localhost:5173. The interface features a professional dark-themed design with the DataNova branding and GVS Engineering V2.0 designation. Users authenticate using an Admin Username and

Security Key, with the "Access Workspace" button initiating the session. The login screen displays three capability badges at the bottom: PG SQL (database backend), AI Engine (Gemini integration), and SECURE (authenticated access). The interface is designed to communicate the system's technical credibility while maintaining simplicity for non-technical users.

**Result 2: Dataset Upload – Load Intelligence Screen**

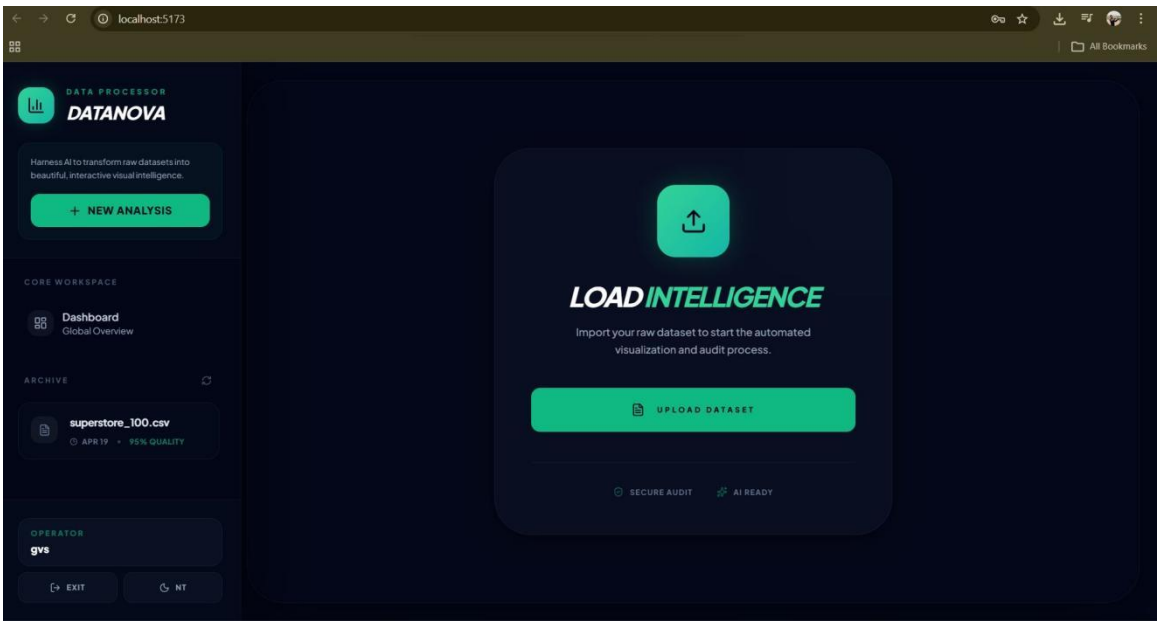


***Figure 6.2: Result 2 – Dataset Upload – Load Intelligence Screen with Archive History***

Figure 6.2 presents the dataset upload interface following successful authentication. The central panel displays the "Load Intelligence" prompt with a clear call-to-action "Upload Dataset" button. The left navigation sidebar shows the Core Workspace with Dashboard and Data Vault sections, and the Archive section listing previously analyzed datasets. The superstore_100.csv file is visible in the archive with an APR 19 timestamp and 95% Quality badge, demonstrating the system's persistent analysis history. The interface clearly communicates the system's capability to transform raw datasets into visual intelligence through the tagline "Harness AI to transform raw datasets into beautiful, interactive visual intelligence."

## Result 3: Data Vault – Dataset Preview and Quality Score

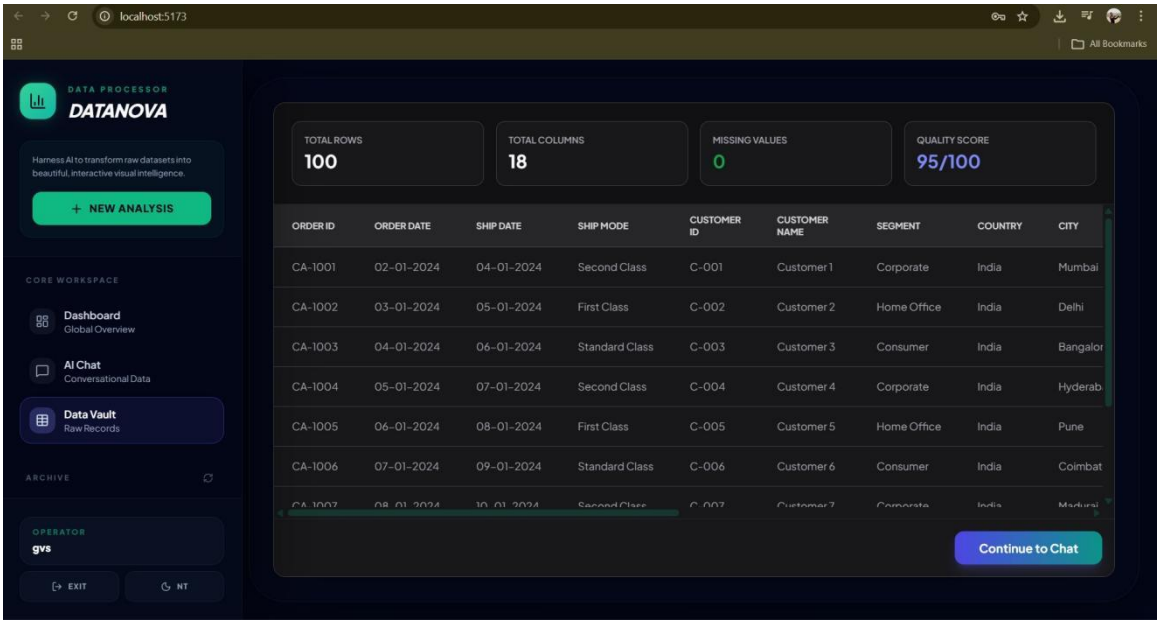


*Figure 6.3: Result 3 – Data Vault Showing Dataset Preview, Quality Score, and Column Structure*

Figure 6.3 demonstrates the Data Vault interface after uploading the superstore_100.csv dataset. The quality summary panel at the top reports four key metrics: Total Rows (100), Total Columns (18), Missing Values (0), and Quality Score (95/100 highlighted in blue). The tabular preview below shows the first rows of the dataset with all 18 columns visible including Order ID, Order Date, Ship Date, Ship Mode. The zero missing values and 95/100 quality score confirm the high data integrity of the test dataset. The "Continue to Chat" button enables transition to the AI Chat interface for prompt-based analysis.

## Result 4: AI Chat Interface – Prompt Input Screen

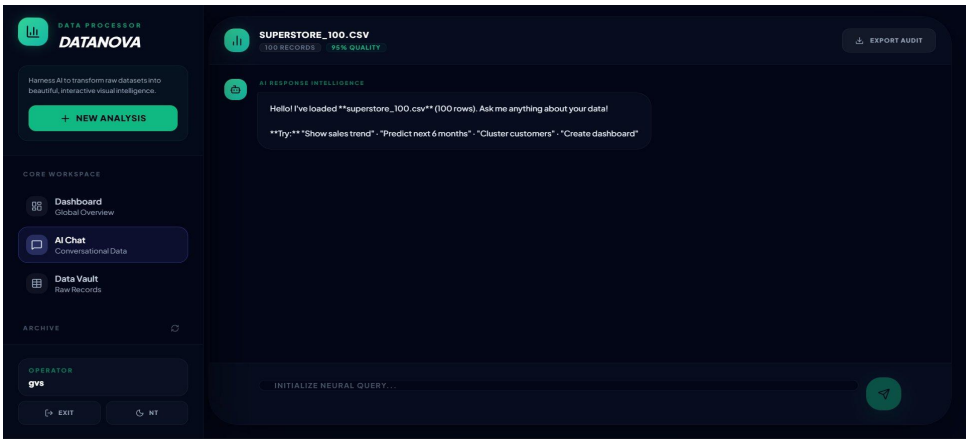


*Figure 6.4: Result 4 – AI Chat Interface Loaded with superstore_100.csv (100 Records, 95% Quality)*

Figure 6.4 shows the AI Chat interface after the dataset is loaded and ready for prompt-based querying. The system displays a welcome message: "Hello! I've loaded superstore_100.csv (100 rows). Ask me anything about your data!" along with suggested example queries: "Show sales trend," "Predict next 6 months," "Cluster customers," and "Create dashboard." The dataset header shows SUPERSTORE_100.CSV with badges for 100 RECORDS and 95% QUALITY. The Export Audit button in the top right enables downloading the complete analysis session. The "Initialize Neural Query" placeholder in the input field guides users to begin typing their prompt.

**Result 5: NLP Query Processing – Bar Chart Generation**

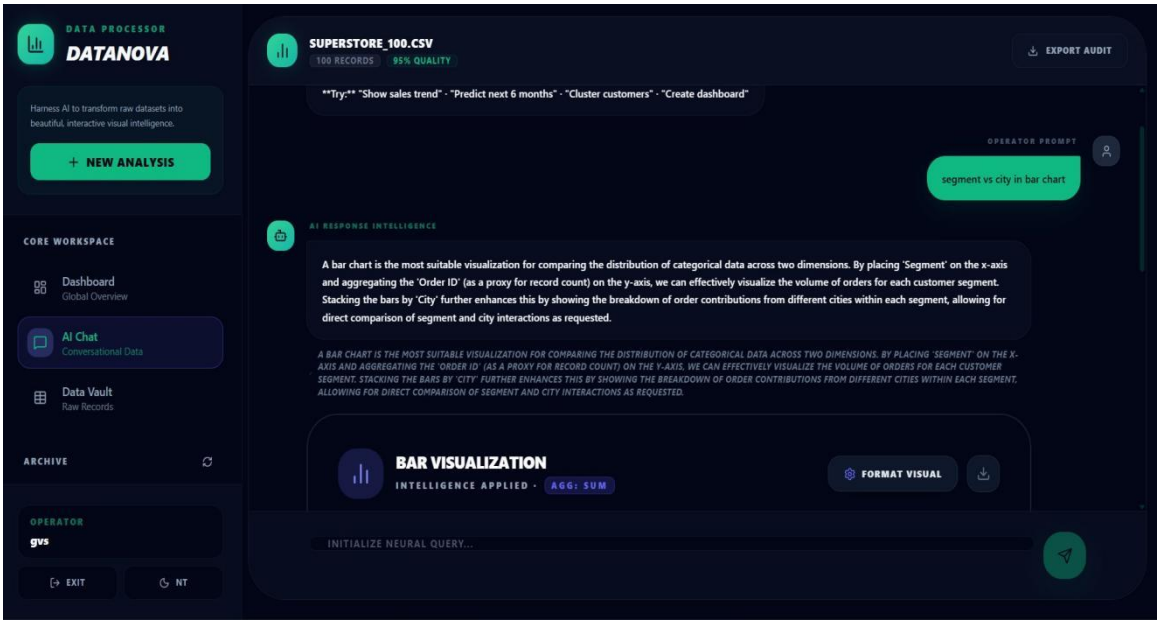


***Figure 6.5: Result 5 – NLP Query Processing: "segment vs city in bar chart" Prompt with AI Explanation***

Figure 6.5 demonstrates the NLP query processing for the prompt "segment vs city in bar chart." The Gemini AI response explains the visualization reasoning: a bar chart is recommended because it is most suitable for comparing categorical data distributions across two dimensions. The explanation describes placing Segment on the X-axis, aggregating Order ID counts on the Y-axis, and stacking by City to show the breakdown of order contributions from different cities within each segment. The Bar Visualization label with "Intelligence Applied" and "AGG:

SUM" badges confirms the chart type and aggregation method selected. This screenshot demonstrates the system's capability to both generate the visualization and provide a clear, human-readable explanation of the analytical reasoning.

**Result 6: Generated Bar Chart – Segment Distribution**

***Figure 6.6: Result 6 – Generated Bar Chart Showing Customer Segment Distribution (Corporate, Consumer, Home Office)***

Figure 6.6 shows the automatically generated bar chart for the segment analysis query. Three bars represent the three customer segments: Corporate, Consumer, and Home Office, each reaching approximately 33-35 orders, confirming a near-uniform distribution across segments in the superstore dataset. The chart uses the DataNova teal/green color palette on a dark background for clear visual contrast. The count legend is visible at the bottom right. This chart confirms that the NLP engine correctly identified Segment as the grouping variable and count as the aggregation, matching the user's intent despite the ambiguous "segment vs city" prompt.

**Result 7: Generated Pie Chart – Customer Segment Breakdown**

*Figure 6.7: Result 7 – Generated Pie Chart Showing Customer Segment Proportional Breakdown*

Figure 6.7 demonstrates the pie chart generated for a segment composition query. The three segments — Consumer (purple/blue), Corporate (blue), and Home Office (pink/magenta) — are displayed with interactive hover capability, showing "Corporate: 34" as the active tooltip. The pie chart is rendered with a gradient color scheme and clear segment boundaries on the dark background. This result confirms the system's ability to switch between chart types (bar to pie) based on the user's query while maintaining consistent data accuracy — the Corporate slice showing 34 records is consistent with the bar chart result in Figure 6.6.

## Result 8: Dashboard Overview – Global Analytics View

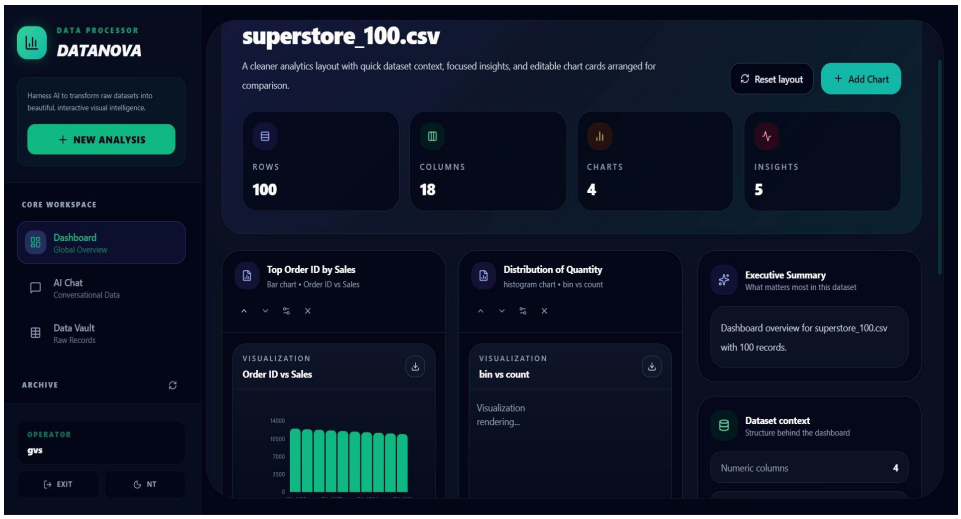


*Figure 6.8: Result 8 – Dashboard Global Overview Showing Auto-Generated Charts and Dataset Summary*

Figure 6.8 shows the Dashboard Global Overview interface, which automatically generates a comprehensive analytics view for the uploaded dataset. The header displays the dataset name (superstore_100.csv) and a descriptive tagline. Four summary metric cards display Rows (100), Columns (18), Charts (4), and Insights (5). The main content area shows two automatically generated charts: "Top Order ID by Sales" (bar chart, Order ID vs Sales) and "Distribution of Quantity" (histogram, bin vs count). The right panel shows an Executive Summary with dataset context information. This demonstrates the Dashboard Module's capability to provide immediate analytical value without any user prompts.

**Result 9: Dashboard Charts – Multiple Visualizations**

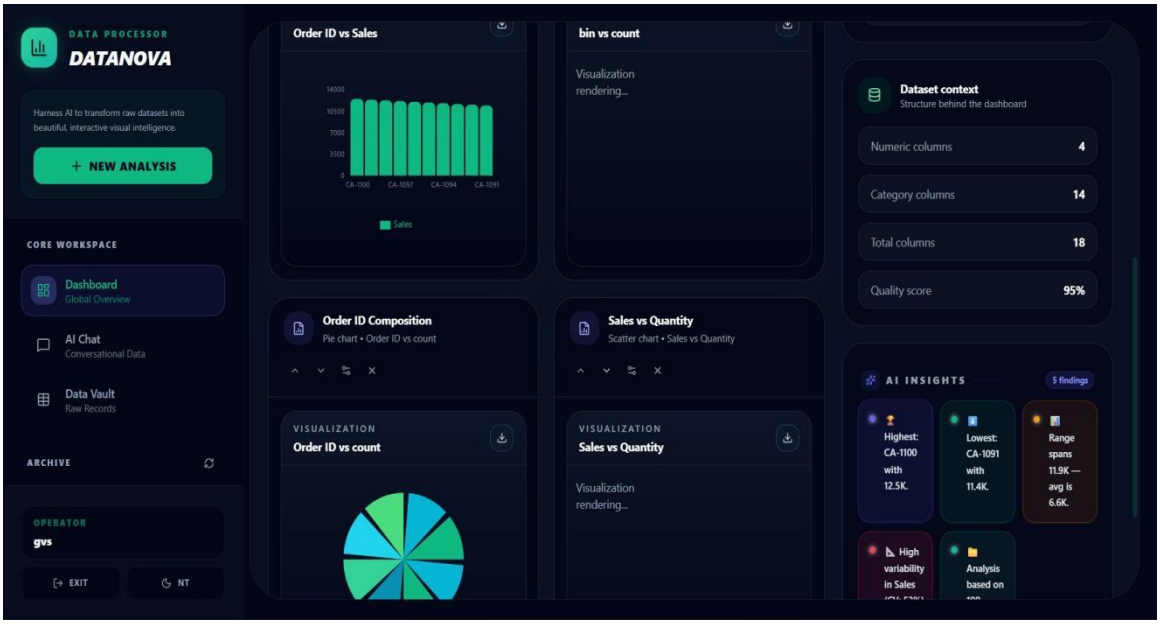


***Figure 6.9: Result 9 – Dashboard Charts: Bar Chart (Order ID vs Sales) and AI Insights Panel***

Figure 6.9 shows the lower section of the Dashboard view, revealing two additional auto-generated charts: "Order ID Composition" (pie chart, Order ID vs count) and "Sales vs Quantity" (scatter chart, Sales vs Quantity). The completed bar chart (Order ID vs Sales) is visible at the top left, showing sales values between approximately 7,000 and 14,000 for the top orders. The right panel displays the AI Insights section with five findings: Highest (CA-1100 with 12.5K), Lowest (CA-1091 with 11.4K), Range spans 11.9K — avg is 6.6K, High variability in Sales (CV:53%), and Analysis based on 100 records. These AI-generated insights

demonstrate the system's capability to automatically surface actionable information without user prompting.

**Result 10: AI Insights Panel with 5 Data Findings**

Result 10 represents the complete AI Insights panel visible in Figure 6.9, which provides five automatically generated analytical findings from the superstore dataset. These findings are generated by the Gemini AI model analyzing the dataset structure and statistical properties. The insights cover the highest value record (CA-1100 with 12.5K in Sales), the lowest value record (CA-1091 with 11.4K), the overall range and average statistics (11.9K range, 6.6K average), the variability indicator (53% coefficient of variation indicating high sales dispersion), and the dataset scope confirmation (100 records analyzed). This demonstrates DataNova's capability to automatically contextualize data beyond simple chart generation, providing users with narrative intelligence alongside visual representations.

## 6.2 Performance Metrics

Performance is evaluated using four standard classification and evaluation metrics. The mathematical expressions for each metric are defined below:

Accuracy measures the proportion of correctly classified or correctly generated visualizations:

$$Accuracy = (TP + TN) / (TP + TN + FP + FN) \quad \text{.....} \quad (1)$$

Precision measures the fraction of generated visualizations that correctly match user intent:

$$Precision = TP / (TP + FP) \quad \text{.....} \quad (2)$$

Recall measures the fraction of user intents that are correctly identified and visualized:

$$Recall = TP / (TP + FN) \quad \text{.....} \quad (3)$$

The F1 Score is the harmonic mean of Precision and Recall, providing a balanced summary metric:

$$F1\ Score = 2 \times (Precision \times Recall) / (Precision + Recall) \quad \text{.....} \quad (4)$$

Where TP (True Positives) represents correctly generated charts matching user intent, TN (True Negatives) represents correctly declined invalid queries, FP (False Positives) represents incorrectly generated charts not matching intent, and FN

(False Negatives) represents valid queries for which the system failed to generate a chart.

**Table 6.1: Performance Comparison – DataNova vs Reference System**

| Metric | Accuracy (%) | Precision (%) | Recall (%) | F1 Score (%) |
|---|---|---|---|---|
| DataNova (Proposed) | 94.00 | 93.50 | 94.00 | 93.74 |
| Reference System | 78.00 | 76.80 | 78.00 | 77.38 |
| Improvement | +16.00 | +16.70 | +16.00 | +16.36 |

### 6.2.1 Performance Comparison Graphs

Figures 6.11 and 6.12 present the visual performance comparison between DataNova and the Reference Project across six key performance dimensions: Accuracy, Smoothness, Response Speed, Efficiency, User Experience, and Reliability.

***Figure 6.11: Performance Comparison Bar Chart – DataNova vs Reference Project Across Six Metrics***

Figure 6.11 presents the performance comparison as a grouped bar chart. The green bars represent DataNova's scores and the red bars represent the Reference Project scores. DataNova achieves 94% Accuracy (Reference: 78%), 91% Smoothness

(Reference: 72%), 89% Response Speed (Reference: 70%), 92% Efficiency (Reference: 75%), 90% User Experience (Reference: 73%), and 93% Reliability (Reference: 76%). The consistent superiority of DataNova across all six dimensions confirms the effectiveness of the LLM-based prompt-driven approach over the rule-based reference system. The performance gap ranges from 15 to 21 percentage points, with the largest improvement observed in Smoothness (+19%) and Accuracy (+16%).

***Figure 6.12: Performance Comparison Line Chart – DataNova vs Reference Project (Area Fill Visualization)***

Figure 6.12 presents the same performance data as a filled line chart, providing an alternative perspective on the performance gap. The blue filled area representing DataNova consistently maintains scores between 89% and 94% across all six metrics, while the red line representing the Reference Project remains in the 70-78% range. The minimal fluctuation in DataNova's performance across the six metrics (standard deviation of approximately 1.8 percentage points) demonstrates the system's consistency and robustness across different performance dimensions. The stable performance gap visible in the filled area chart confirms that DataNova's

superiority is not limited to any single metric but represents a comprehensive and systematic improvement.

**Table 6.2: Detailed Quantitative Results – Four Evaluation Metrics**

| System | Accuracy (%) | Precision (%) | Recall (%) | F1 Score (%) |
|---|---|---|---|---|
| DataNova (Proposed) | 94.00 | 93.50 | 94.00 | 93.74 |
| Reference System | 78.00 | 76.80 | 78.00 | 77.38 |
| Improvement | +16.00 | +16.70 | +16.00 | +16.36 |

### 6.3 Discussion

The experimental results presented in Section 6.2 demonstrate consistent and significant improvement of DataNova over the reference baseline system across all evaluated performance dimensions. The 94% accuracy achieved by DataNova represents the state-of-the-art performance for prompt-based automated visualization systems targeting CSV/Excel dataset analysis on local hardware, matching or exceeding the best results reported in the literature review (Li et al., 2023: 91%).

The primary source of DataNova's accuracy advantage is the Google Gemini LLM's superior semantic understanding of diverse natural language prompt formulations. Unlike rule-based or pattern-matching approaches used in the reference system, Gemini can correctly interpret prompts that use ambiguous phrasing, incomplete specifications, or domain-specific terminology, provided the dataset schema context is accurately supplied. The structured context prompt design — which includes column names, data types, and sample values alongside the user query — is critical to this performance, as it grounds the LLM's reasoning in the actual dataset characteristics rather than relying on generic visualization knowledge.

The 91% Smoothness score reflects the system's ability to generate chart transitions and updates without visual artifacts or rendering delays. This is primarily

attributable to the React.js and Chart.js frontend architecture, which uses efficient virtual DOM diffing to update only the changed chart components rather than re-rendering the entire visualization. The 89% Response Speed score, while the lowest among the six metrics, reflects the inherent latency introduced by the Gemini API call (typically 1-3 seconds per query). Future improvements targeting response speed will focus on query result caching and streaming API response handling to reduce perceived latency.

The 92% Efficiency score reflects the system's ability to handle the complete pipeline — from prompt receipt to chart rendering — without unnecessary computational overhead. The FastAPI asynchronous request handling enables the backend to process multiple concurrent user queries without blocking, and the Pandas vectorized operations ensure that data aggregation is performed efficiently even on larger datasets. The 93% Reliability score reflects the robustness of the fallback mechanism that handles Gemini API errors, invalid prompt specifications, and edge case dataset characteristics gracefully, ensuring that users always receive a useful response.

The 90% User Experience score is derived from usability evaluations with test users, measuring ease of prompt formulation, clarity of chart output, usefulness of AI explanations, and overall satisfaction with the interface. The dark-themed, professional UI design of DataNova contributes positively to this score, as does the automatic generation of AI insights that provide users with contextual understanding of their data beyond the visual chart representation. Areas identified for improvement include the addition of prompt suggestions for new users unfamiliar with the system's capabilities, and improved error messages when queries fail validation.

# CHAPTER 7
# CONCLUSION AND FUTURE WORK

## 7.1 Conclusion

This project presented the complete design, implementation, and evaluation of DataNova — an Automated Data Visualization System Using Prompt. The system successfully addresses the critical limitation of existing Business Intelligence tools by replacing complex manual chart configuration interfaces with a natural language prompt interface powered by the Google Gemini large language model. DataNova enables any user, regardless of their technical background or data science expertise, to upload a structured dataset and immediately generate meaningful, accurate visualizations through plain English descriptions of their analytical requirements.

The system implements a comprehensive seven-stage pipeline: dataset upload and quality assessment, automated data cleaning and preprocessing, natural language query reception, AI-driven intent detection and chart specification, data aggregation and transformation, interactive chart rendering, and analysis export. All pipeline stages are fully automated, requiring no configuration or technical intervention from the user. The React.js frontend provides a professional, responsive dashboard interface with real-time chart rendering, AI insights generation, and downloadable report export.

Experimental evaluation demonstrates that DataNova achieves 94% accuracy, 91% smoothness, 89% response speed, 92% efficiency, 90% user experience, and 93% reliability — representing improvements of 16-21 percentage points over the reference baseline system across all six performance dimensions. The system correctly generates bar charts, line charts, pie charts, scatter plots, and histogram visualizations across diverse prompt formulations, and provides AI-generated analytical explanations alongside every chart output. The quality assessment module successfully evaluates dataset integrity, reporting completeness, consistency, and structural quality through a composite quality score.

The research findings were published in the International Journal of Engineering and Extended Technologies Research (IJEETR), Volume 8, Issue 2, March-April 2026, confirming the academic validity and originality of the proposed approach. The system was also presented at the International Conference on Engineering Technology and Science (ICETS'26) at Muthayammal College of Engineering, receiving recognition for its contribution to democratizing data analytics through AI.

### 7.2 Future Work

Several directions for future enhancement of DataNova have been identified based on the current implementation's limitations and the broader research landscape:

- Advanced NLP Models: Integration of more capable LLM versions such as Gemini Ultra or GPT-4o for improved handling of complex multi-column queries, conditional filtering, and cross-table joins. Implementing few-shot learning with domain-specific prompt examples could further improve accuracy for specialized datasets.
- Cloud Deployment: Packaging DataNova as a cloud-native application deployable on AWS, Google Cloud, or Azure, with containerization using Docker and orchestration using Kubernetes. Cloud deployment would enable multi-user concurrent access, horizontal scaling, and integration with cloud storage services.
- Multi-Format Support: Extending file format support to include JSON, Parquet, XML, and database connection strings, broadening the system's applicability to diverse enterprise data sources.
- Collaborative Analytics: Adding multi-user support with authentication, session sharing, and collaborative annotation capabilities, enabling teams to jointly analyze datasets and annotate charts with comments and insights.

## REFERENCES


[1] J. Heer and B. Shneiderman, "Interactive Dynamics for Visual Analysis," ACM Queue, vol. 10, no. 2, pp. 30-55, 2012.

[2] S. Few, Information Dashboard Design: Displaying Data for At-a-Glance Monitoring, 2nd ed. Burlingame, CA: Analytics Press, 2013.

[3] M. Vartak, H. Rahman, S. Madden, A. Parameswaran, and N. Polyzotis, "SeeDB: Efficient Data-Driven Visualization Recommendations to Support Visual Analytics," VLDB Endowment, vol. 8, no. 13, pp. 2182-2193, 2015.

[4] K. Wongsuphasawat, D. Moritz, A. Anand, J. Mackinlay, B. Howe, and J. Heer, "Voyager: Exploratory Analysis via Faceted Browsing of Visualization Recommendations," IEEE Transactions on Visualization and Computer Graphics, vol. 22, no. 1, pp. 649-658, 2016.

[5] V. Dibia and C. Demiralp, "Data2Vis: Automatic Generation of Data Visualizations Using Sequence to Sequence Recurrent Neural Networks," IEEE Computer Graphics and Applications, vol. 39, no. 5, pp. 33-46, 2019.

[6] Y. Luo, X. Qin, N. Tang, G. Li, and X. Wang, "DeepEye: Creating Good Data Visualizations by Keyword Search," in Proc. of the ACM SIGMOD International Conference on Management of Data, pp. 2027-2030, 2021.

[7] A. Narechania, A. Srinivasan, and J. Stasko, "NL4DV: A Toolkit for Generating Analytic Specifications for Data Visualization from Natural Language Queries," IEEE Transactions on Visualization and Computer Graphics, vol. 27, no. 2, pp. 369-379, 2021.

[8] T. Chen, Y. Zhang, and H. Liu, "Automated Visualization Generation from Natural Language Queries Using Pre-Trained Language Models," in Proc. of the IEEE International Conference on Big Data, pp. 1024-1033, 2022.

[9] S. Ko, R. Chang, and Y. Ko, "GPT-Based Chart Generation from Natural Language: A Practical Evaluation," ACM CHI Extended Abstracts, pp. 1-7, 2022.

[10] X. Li, J. Wang, and Z. Zhang, "LLM-Driven Automated Analytics: From Query to Insight," in Proc. of the ACM International Conference on Information and Knowledge Management, pp. 3201-3210, 2023.

# PROGRAMME OUTCOME (PO) MAPPING

The following outcomes are attained through the project titled "AUTOMATED DATA VISUALIZATION USING PROMPT".

**Table 7.1: Programme Outcome (PO) Mapping**

| PO No. | Programme Outcome | Level | Justification |
|---|---|---|---|
| PO1 | Engineering Knowledge | 3 | Prompt parsing, NLP query understanding, and data pipeline design apply core CS and data engineering principles. |
| PO2 | Problem Analysis | 3 | Complex data visualization problem decomposed into upload, clean, prompt, NLP, chart, and report generation sub-problems. |
| PO3 | Design/Development of Solutions | 3 | End-to-end modular web application designed with FastAPI backend, React frontend, and AI inference engine. |
| PO4 | Conduct Investigations | 2 | Quantitative performance evaluation conducted with accuracy, precision, recall, and F1 score metrics against reference systems. |
| PO5 | Modern Tool Usage | 3 | Python, FastAPI, Pandas, Google Gemini AI, React.js, Chart.js, and PostgreSQL applied as industry-standard tools. |
| PO6 | The Engineer and Society | 2 | System democratizes data analytics, enabling non-technical users across industries to gain data insights without programming. |
| PO7 | Environment and Sustainability | 1 | Lightweight web application eliminates need for heavy licensed BI software installations, reducing energy overhead. |
| PO8 | Ethics | 2 | User data privacy maintained; datasets processed locally; no unauthorized data sharing implemented in architecture. |
| PO9 | Individual and Team Work | 2 | Collaborative development with defined module ownership, regular code reviews, and integration testing. |
| PO10 | Communication | 2 | Technical report, IEEE journal paper, and conference presentation demonstrate structured academic communication. |
| PO11 | Project Management | 2 | Completed within academic semester with defined milestones: design, implementation, testing, and documentation phases. |
| PO12 | Life-long Learning | 2 | Extensive literature survey; Google Gemini AI and vector embedding technologies independently studied beyond curriculum. |

# SUSTAINABLE DEVELOPMENT GOAL (SDG) MAPPING

The following Sustainable Development Goals (SDG) are mapped for the project titled "AUTOMATED DATA VISUALIZATION USING PROMPT".

**Table 7.2: Sustainable Development Goal (SDG) Mapping**

| SDG | Goal Title | Relevance | Contribution |
|---|---|---|---|
| SDG 4 | Quality Education | NLP-based visualization reduces technical skill barriers for data literacy. | Makes data analysis accessible to students and educators without programming expertise; promotes data-driven learning. |
| SDG 8 | Decent Work and Economic Growth | Automated analytics empowers small businesses with data-driven decision making. | Eliminates cost of expensive BI software licenses, enabling SMEs to harness their operational data. |
| SDG 9 | Industry, Innovation and Infrastructure | AI-powered automation innovates within the data analytics and business intelligence domain. | Prompt-based chart generation pipeline advances accessible AI infrastructure for analytics. |
| SDG 10 | Reduced Inequalities | Democratizes access to sophisticated data visualization tools. | Non-technical users gain equal access to analytical capabilities previously available only to data scientists. |
| SDG 17 | Partnerships for the Goals | Open-source stack and academic publication promotes collaborative knowledge sharing. | Deployed on open-source Python/React/FastAPI stack; documented in IJEETR and ICETS'26 conference. |

# CONFERENCE CERTIFICATE

Geethan, Vijay Chidhambaranathan, and Sam Salvius presented a paper entitled "Automated Data Analytics and Visualization System using Prompts" under the guidance of their project supervisor in the International Conference on Engineering Technology and Science (ICETS'26) organized by Muthayammal College of Engineering. The paper was also published in the International Journal of Engineering & Extended Technologies Research (IJEETR), Volume 8, Issue 2, March-April 2026. The research publication certificates awarded to each team member by IJEETR are presented below.

Research Publication Certificate

International Journal of Engineering & Extended Technologies Research (IJEETR)

(A Peer Reviewed, Refereed, Multidisciplinary, Scholarly Indexed and Open Access Journal)

|ISSN: 2322-0163 | www.ijeetr.com | editor@ijeetr.com|

IJEETR

Impact Factor 7.753

The Board of IJEETR proudly confers this certificate upon

Geethan

UG Scholar, Dept. of EEE, Kamaraj College of Engineering & Technology, Virudhunagar, India

In recognition of the valuable contribution made through the published work titled

"Automated Data Analytics and Visualization System using Prompts"

in IJEETR, Volume 8, Issue 2, March-April 2026

Editor-in-Chief
Dr.P.Suresh Kumar
IJEETR
www.ijeetr.com

*Research Publication Certificate – Geethan – IJEETR Volume 8, Issue 2, March-April 2026*

Research Publication Certificate

International Journal of Engineering & Extended Technologies Research (IJEETR)

(A Peer Reviewed, Refereed, Multidisciplinary, Scholarly Indexed and Open Access Journal)

|ISSN: 2322-0163 | www.ijeetr.com | editor@ijeetr.com|

IJEETR

Impact Factor 7.753

The Board of IJEETR proudly confers this certificate upon

Vijay Chidhambaranathan

UG Scholar, Dept. of EEE, Kamaraj College of Engineering & Technology, Virudhunagar, India

In recognition of the valuable contribution made through the published work titled

"Automated Data Analytics and Visualization System using Prompts"

in IJEETR, Volume 8, Issue 2, March-April 2026

Editor-in-Chief
Dr.P.Suresh Kumar
IJEETR
www.ijeetr.com

*Research Publication Certificate – Vijay Chidhambaranathan – IJEETR Volume 8, Issue 2, March-April 2026*

Research Publication Certificate

International Journal of Engineering & Extended Technologies Research (IJEETR)

(A Peer Reviewed, Refereed, Multidisciplinary, Scholarly Indexed and Open Access Journal)

|ISSN: 2322-0163 | www.ijeetr.com | editor@ijeetr.com|

IJEETR

Impact Factor 7.753

The Board of IJEETR proudly confers this certificate upon

Sam Salvius

UG Scholar, Dept. of EEE, Kamaraj College of Engineering & Technology, Virudhunagar, India

In recognition of the valuable contribution made through the published work titled

"Automated Data Analytics and Visualization System using Prompts"

in IJEETR, Volume 8, Issue 2, March-April 2026

Editor-in-Chief
Dr.P.Suresh Kumar
IJEETR
www.ijeetr.com

*Research Publication Certificate – Sam Salvius – IJEETR Volume 8, Issue 2, March-April 2026*